National University of Computer and Emerging Sciences
Chiniot-Faisalabad Campus

Programming Fundamentals (CS1002)
Final Exam Subjective Solution
Fall 2025

Note: Any questions attempted using non-permanent (erasable) pens will not be marked.

Attempt all the questions.

CLO 1: Describe fundamental concepts of structured and procedural programming, use pseudo-codes and simple programs to understand control structures, iterative structures and functions using C++ language.

Q1: 4+3+2+7+5 = [20 marks]

Attempt this question on the provided answer sheet.

Write down different functions only that fulfil the requirements given in each part.

Note: Do not try to write a complete code if not asked. This question is just meant to evaluate your understanding of functions.

a. Write down two overloaded functions named as **areaAndBorder**. One of these will have one double value as parameter and will somehow return the area and circumference of the circle. The other function will have two double parameters and will return the area and perimeter of the rectangle.

Solution:

```
void areaAndBorder(double radius, double &area, double &circumference)
{
    area = 3.14 * radius * radius;
    circumference = 2 * 3.14 * radius
}
void areaAndBorder(double length, double width, double &area, double &perimeter)
{
    area = length * width;
    perimeter = 2 * (length + width);
}
```

(4 marks)

(**Note:** Area of circle = 3.14 * radius * radius, circumference = 2 * 3.14 * radius)

b. Write down **only one** function that can take either two parameter, three parameters or four parameters and will return the product of the numbers passed to it. Data type of the parameters must be double. **(3 marks)**

Solution:

```
double product(double a, double b, double c = 1.0, double d = 1.0)
{
    return a * b * c * d;
}
```

c. Provided the three overloaded functions prototypes below: **(2 marks)**

```
int compute(int, int = 20);
double compute(double);
float compute(float, float);
```

Give one function call that is ambiguous and will generate a compiler error and two function calls that will not generate any error.

Solution:

Ambiguous: compute(10);

Non Ambiguous: compute(10, 30); compute(2.5f, 3.5f);

d. Write down a function that takes two arrays of same size as parameters. One of type int and other of type float namely intArray and floatArray respectively. This function will read a number from the user in each function call. This number must be a positive integer or a positive float value. The number will then be checked, if it is integer, then it will be stored in intArray otherwise in floatArray at the indexes passed to it (It will always be after the last value). The function will print the count of float values and integer values separately, input by the user so far in the program, including this call. This count must be maintained locally. (Do not use global variables) **(7 marks)**

Solution:

```
void storeNumber(int intArray[], float floatArray[], int& intIndex, int& floatIndex)
{
    static int intCount = 0;
    static int floatCount = 0;
    double num;
    cout << "Enter a positive number: ";
    cin >> num;
    if (num > 0 && num == static_cast<int>(num))
    {
            intArray[intIndex++] = static_cast<int>(num);
            intCount++;
    }
    else if (num > 0)
    {
            floatArray[floatIndex++] = static_cast<float>(num);
            floatCount++;
    }
    cout << "Total Integers so far: " << intCount << endl;
    cout << "Total Floats so far: " << floatCount << endl;
}
```

e. Write down a small driver program that will call the function written in the above Part d, thirty (30) times. After the input of thirty numbers the valid values from both the arrays will be printed. Keep the track of count of numbers added to any of the arrays, so that to pass the correct index for each array at which the next number is to be inserted. This will definitely be updated inside the function. **(4 marks)**

Hint: Think of pass by value, pass by reference and static variables

SOLUTION

```
int main()
{
        const int SIZE = 30;
        int intArray[SIZE];
        float floatArray[SIZE];
        int intIn = 0;
        int floatIn = 0;
        for (int i = 0; i < SIZE; i++)
        {
                storeNumber(intArray, floatArray, intIn,floatIn);
        }
        cout << "\nValid Integer Values:\n";
        for (int i = 0; i < intIn; i++)
        {
                cout << intArray[i] << " ";
        }
        cout << "\nValid Float Values:\n";
        for (int i = 0; i < floatIn; i++)
        {
                cout << floatArray[i] << " ";
        }
        return 0;
}
```

CLO 1: Describe fundamental concepts of structured and procedural programming, use pseudo-codes and simple programs to understand control structures, iterative structures and functions using C++ language.

Q2: [20 marks]

Attempt this question on the provided answer sheet.

You are working on a rover landing on Mars. The rover uses a 2D grid of sensors to measure ground elevation. However, due to radiation, the raw data from the sensors is "noisy" (it has random spikes). To fix this, you need to apply a **smoothing filter** to the data before the rover can decide where to drive.

The Problem: Write a function smooth_terrain in C++ that takes a raw 2D array of sensor data and produces a processed 2D array.

```
const int ROWS = 5;
const int COLS = 5;
void smooth_terrain(int raw_data[ROWS][COLS], int processed_data[ROWS][COLS]);
//you have to write the definition of this function
```

The Logic:
The value of any cell (r, c) in the processed array should be the average of the cell itself and its valid immediate neighbors (Up, Down, Left, Right).
NewValue(r,c) = Sum of (Cell + Valid Neighbors) / Count of (Cell + Valid Neighbors)

Constraints & Rules:

1. Input: A 2D integer array raw_data[ROWS][COLS].
2. Output: Store the result in processed_data[ROWS][COLS].
3. Boundaries: You must handle the edges of the grid correctly. For example, a corner cell only has 2 neighbors; a side cell has 3. Do not access indices out of bounds.
4. Integer Math: Use integer division (floor) for the average.

Example: Input Grid (3x3):

10	20	10
10	50	10
10	10	10

Consider this calculation for the center pixel (50): Valid neighbors are 20 (up), 10 (left), 10 (right), 10 (down). Sum = 50 + 20 + 10 + 10 + 10 = 100. Count = 5. New Value = 100 / 5 = 20.

Now update that value in processed array at same index

	20	

SOLUTION

```
void smooth_terrain(int raw_data[ROWS][COLS], int processed_data[ROWS][COLS])
{
	for (int r = 0; r < ROWS; r++)
	{
		for (int c = 0; c < COLS; c++)
		{
			int sum = raw_data[r][c];
			int count = 1;

			// Up
			if (r - 1 >= 0)
			{
			sum += raw_data[r - 1][c];
			count++;
			}

			// Down
			if (r + 1 < ROWS)
			{
			sum += raw_data[r + 1][c];
			count++;
			}

			// Left
			if (c - 1 >= 0)
			{
			sum += raw_data[r][c - 1];
			count++;
			}

			// Right
			if (c + 1 < COLS)
			{
			sum += raw_data[r][c + 1];
			count++;
			}
			processed_data[r][c] = sum / count;  // Integer division
		}
	}
}
```

CLO 1: Describe fundamental concepts of structured and procedural programming, use pseudo-codes and simple programs to understand control structures, iterative structures and functions using C++ language.

Q3: [20 marks]

Attempt this question on the provided answer sheet.

Your class instructor maintains daily attendance records in a text file named "**attendance.txt**". Each line in the file contains: a roll number, a space, and a sequence of 22 characters, each representing attendance for one day of the month where 'P' represents Present, 'A' represents Absent, and 'L' represents Late. However, the count of total number of student records is unknown. A sample input file is:

F250501	PPAPPAAPPPLPPAAAPAAPAA
F250509	PPPPLLPPLPPPPLPPPLPLPP
F250541	PPAPPPAAAAAPPLLLPAAAAL

Write a C++ program that reads the attendance data from attendance.txt. And, for each student, calculates the total number of Present, Absent, and Late days. Also, calculate the attendance percentage for each student as:

$$Attendance\ \% = \frac{Present\ Days + 0.5\ X\ Late\ Days}{22}\ X\ 100$$

Finally, write the results into an output file **"report.txt"** in the following format:

Roll_No	Present	Absent	Late	Attendance%
F250501	11	10	1	52.27
F250509	16	0	6	86.36
F250541	8	10	4	45.45

Generate another output file "**warnings.txt**" containing Roll No. of all the students (each separated by a comma) with attendance% below 75; a sample file is shown.

F250501, F250541,

Use strings; your solution must include all the necessary header files, and code for file handling Errors.

SOLUTION

Q4: [20 marks]

Write a C++ program that takes an integer **N** as input and prints all **Automorphic Numbers** between **1 and N** (inclusive).

A number is Automorphic if its square ends with the number itself.

Examples:

- 5 → 25 → ends with 5 → Automorphic
- 6 → 36 → ends with 6 → Automorphic
- 25 → 625 → ends with 25
- 76 → 5776 → ends with 76

This question is to be attempted below, partial code is already given, complete the rest of the code:

```
#include <iostream>
using namespace std;
int main()
{
    int N;
    cout << "Please (N) up till which you want to print the Automorphic Numbers: ";
    cin >> N; // Focus on the logic only, input validation is not required
    // Attempt this question below. Partial code is given; complete the rest.
 ------------------------------------------------------------------------------
    SOLUTION


    cout << "Automorphic Numbers between 1 and " << N << " are: " << endl;

    for (int i = 1; i <= N; i++) // loop from 1 to N +5 marks
    {
        int square = i * i;
        int temp = i;   // Store number in temp variable
        bool isAuto = true; // Reset the bool variable to true every time

        while (temp > 0)
        {
            if (temp % 10 != square % 10) // Comparing the last digit of both numbers
            {
                isAuto = false;
                break;
            }
            temp /= 10;
            square /= 10;
        }
        // Correct logic for Automorphic numbers +15 marks

        if (isAuto) // Printing
        {
            cout << i << " ";
        }
    }

    return 0;
}
```

ACCEPTABLE ALTERNATE

```
#include <iostream>
#include <cmath>
using namespace std;

int main()
{
	int N;
	cout << "Please (N) up till which you want to print the Automorphic Numbers: ";
	cin >> N;

	cout << "Automorphic Numbers between 1 and " << N << " are: " << endl;

	for (int i = 1; i <= N; i++)   // loop from 1 to N
	{
		int temp = i;
		int digitCount = 0;

		// Step 1: Count digits of i
		while (temp > 0)
		{
			digitCount++;
			 temp /= 10;
		}

		// Step 2: Calculate square of i
		int square = i * i;

		// Step 3: Calculate 10^digitCount
		int power = pow(10, digitCount);

		// Step 4: Extract last digits from square
		int extracted = square % power;

		// Step 5: Compare
		if (extracted == i)
		{
			 cout << i << " ";
		}
	}

	return 0;
}
```

❌ Hardcoded digit counting using ranges, NOT ACCEPTABLE

```
#include <iostream>
#include <cmath>
using namespace std;

int main()
{
	int N;
	cout << "Please (N) up till which you want to print the Automorphic Numbers: ";
	cin >> N;

	cout << "Automorphic Numbers between 1 and " << N << " are: " << endl;

	for (int i = 1; i <= N; i++)
	{
		int digitCount;

		// ❌ Hardcoded digit counting using ranges
		if (i >= 1 && i <= 9)
			digitCount = 1;
		else if (i >= 10 && i <= 99)
			digitCount = 2;
		else if (i >= 100 && i <= 999)
			digitCount = 3;
		else if (i >= 1000 && i <= 9999)
			digitCount = 4;
		else
			digitCount = 5;   // Assumption (bad practice)

		int square = i * i;
		int power = pow(10, digitCount);

		int extracted = square % power;

		if (extracted == i)
		{
			 cout << i << " ";
		}
	}

	return 0;
}
```